

Cody: An LLM-Augmented Static Analysis Tool for Interactive Codebase Comprehension

Author: Sahil Gupta | **Date:** July 2026 | **Repository:** github.com/sahilgupta2006/Cody

1. The Problem: Why Codebase Comprehension Needs Better Tooling

Understanding unfamiliar source code is the single largest time-sink in professional software development. Three independent empirical studies quantify this:

- **Xia et al. (IEEE TSE, 2018)** conducted a large-scale field study with 78 professional developers and found they spend **58% of their working time** on program comprehension activities -- reading, navigating, and understanding existing code -- rather than writing new code [1].

- **Ko et al. (IEEE ICSE, 2006)** observed developers in situ and reported they spend **35% of their time solely on code navigation** -- jumping between files, tracing call chains, and locating relevant definitions [2].

- **Minelli et al. (IEEE ICPC, 2015)** analyzed 750+ hours of IDE interaction data and found that developers switch context an average of **13 times per minute** when exploring unfamiliar codebases, with navigation consuming more effort than editing [3].

Despite this, the tools available today offer fragmented solutions:

Tool Category	What It Does	What It Misses
IDE "Go to Definition"	Jumps to one symbol ..	No holistic map; requires knowing *what* to search for
grep / ripgrep	Raw text search	No semantic understanding of call relationships
Static call graph tools (Py..)	Generates call graphs	Misses dynamic dispatch (callbacks, thread targets);..
AI chat (ChatGPT, Copilot)	Free-form Q&A about ..	Not grounded in the actual repo; hallucinates withou..
Code intelligence (Sourcegr..)	Full indexing and na..	Commercial SaaS; requires cloud infrastructure and t..

The core gap: No existing tool combines *structural analysis* (who calls whom) with *natural-language explanations* (what does this function do) in a *guided, visual walkthrough* that runs fully offline on the developer's machine.

Cody fills this gap.

2. What Cody Does

Cody takes any Git repository URL as input and produces an **interactive dependency graph** with a guided walkthrough. Concretely, a user can:

1. **Paste a GitHub URL** and click Analyze.
2. **See a force-directed graph** of all functions and classes, with edges showing who-calls-whom.
3. **Step through the codebase** using a BFS-ordered walkthrough that starts from entry points (e.g., `main()`), with the graph growing organically at each step.

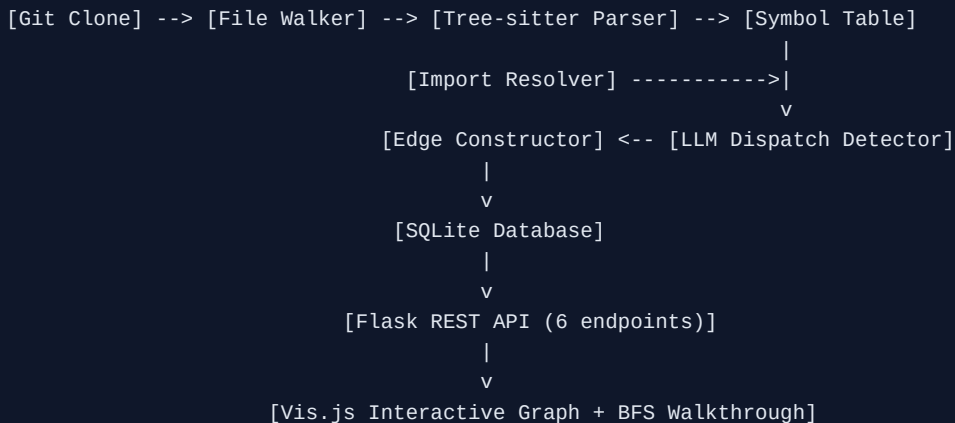
4. Click any **node** to jump to it, see its source code, and get an LLM-generated plain-English explanation.

5. Chat with the AI about any specific function, with the LLM's context grounded in that function's actual source code.

All LLM inference runs locally via Ollama (Qwen2.5-Coder:3b). No code leaves the developer's machine.

3. How It Works: System Architecture

The system has four layers, implemented in ~3,000 lines across Python (backend) and JavaScript (frontend):



3.1 Pass 1: Symbol Extraction (parser.py, analyzer.py)

Cody walks the cloned repository with `os.walk()`, reads every source file, and parses it using **Tree-sitter** -- an incremental parsing framework that generates concrete syntax trees. For each file:

- The parser identifies **function definitions** and **class definitions** based on language-specific AST node types (e.g., `function_definition` for Python, `method_declaration` for Java, `function_item` for Rust).
- Each symbol is recorded as a **Node** with its name, type, source location (line numbers + byte offsets), file path, and hierarchical scope (e.g., `ClassName:method_name`).
- A **scope table** maps (`filepath`, `scope_path`) to node IDs, enabling call-site resolution in Pass 2.

This supports **10+ languages** (Python, Rust, Go, Java, JavaScript, TypeScript, C++, C#, and variants) through a unified configuration map. A regex-based fallback handles languages without installed Tree-sitter bindings.

For Python files, an **import resolution engine** also runs during this pass. It parses `import` and `from...import` AST nodes and resolves them to concrete file paths within the repository -- handling relative imports (`from ..utils import helper`), aliased imports (`import X as Y`), and package `__init__.py` resolution.

3.2 Pass 2: Edge Construction (analyzer.py)

The second pass re-traverses every file's syntax tree, this time looking for **call expressions**. For each call site:

1. The **caller** is identified via the scope table (which function contains this call?).
2. The **callee name** is extracted from the call AST node.
3. The callee is resolved using a **5-tier priority cascade**:

Priority	Strategy	Example
1	Import lookup	<code>`from utils import helper`</code> -- resolves <code>`helper`</code> to <code>`utils.py`</code>
2	Same-file match	Call to <code>`process()`</code> resolves to <code>`process()`</code> defined in the same file
3	Globally unique	If only one function named <code>`run_server`</code> exists in the entire repo, use it
4	Same-package	Among multiple candidates, prefer the one in the same directory
5	Fallback	Use the first candidate found

If no match is found in the codebase, the call is recorded as a **library entity** (e.g., `library_entity:print`)

LLM-Augmented Dynamic Dispatch Detection: Static analysis cannot detect function pointers, callbacks, or thread targets. When a call expression's arguments contain identifiers that match known function names (e.g., `Thread(target=worker_fn)`), Cody sends the code snippet to a local LLM:

```
Prompt: "Is this line invoking a user-defined function as callback or thread target?
        If yes return just the function name. If no return NULL."
Input:  threading.Thread(target=worker_fn, daemon=True).start()
Output: worker_fn
```

The detected function is added as a **callback** or **thread_target** edge. This is a **novel contribution** -- using local LLM inference at analysis time to augment a static call graph with edges invisible to traditional static analyzers.

3.3 Persistence & API (database.py, app.py)

The resulting graph (nodes + typed edges) is stored in a **SQLite database** with two tables:

- `nodes(id, name, type, hierarchical_name, start_row, end_row, filepath)`
- `edges(from_node, to_node, type)` -- where type is `direct`, `callback`, or `thread_target`

A Flask server exposes 6 REST endpoints: analyze a repo, fetch the graph, get node details/source code, get LLM explanations, contextual chat, and compute walkthrough order.

3.4 Interactive Visualization (app.js)

The frontend renders the graph using **Vis.js** with several UX innovations:

- **Accumulative focal subgraph:** Instead of showing the entire graph at once (overwhelming for large codebases), nodes appear progressively as the user steps through the walkthrough. Each step reveals the current node plus its immediate callees.
- **Animated node placement:** New nodes slide out from their parent using linear interpolation (LERP), with collision detection to prevent overlap.
- **BFS walkthrough engine:** The server computes a breadth-first traversal starting from automatically detected entry points (zero in-degree nodes, prioritizing `main/cli`). This mirrors the natural "top-down comprehension" cognitive strategy described by Brooks (1983) [4].
- **Color-coded semantics:** Active node glows gold; outbound callees are rose; thread targets are green; callbacks are red; library calls are dashed slate.
- **Click-to-jump:** Clicking any node recomputes the BFS from that node and updates the walkthrough.

- **Per-node chat:** An "Ask Cody" drawer opens a chat where the LLM's system prompt includes the function's actual source code, ensuring grounded (not hallucinated) responses.

4. How Cody Compares

Capability	PyCG [5]	Sourcetrail	Copilot Chat	**Cody**
Multi-language AST parsing	Python only	C/C++, Java, Python	Via LLM	**10+ languages**
Dynamic dispatch detection	No	No	Partial (no graph)	**Yes (LLM-augmented)**
Interactive graph visualization	No	Yes	No	**Yes**
Guided BFS walkthrough	No	No	No	**Yes**
Per-node LLM explanations	No	No	Yes (ungrounded)	**Yes (code-grounded)**
Fully offline / privacy-preserving	Yes	Yes	No (cloud)	**Yes**
Zero setup (no plugins/servers)	Yes	No	No	**Yes**

5. Current Scope & Next Steps

Current implementation: 3,054 lines of code across 10 source files. 21-page detailed technical documentation available in the repository. Tested on Python repositories with standard import patterns.

Limitations: Import resolution currently Python-only for cross-file accuracy; other languages fall back to name matching. No type inference for polymorphic dispatch. No incremental analysis (full re-parse on each run).

Planned extensions: Incremental file-hash caching, multi-language import resolvers (Java classpath, Go modules, JS/TS node_modules), VS Code extension packaging, and graph diff mode for comparing dependency structures across Git branches.

References

- [1] X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, and S. Li, "Measuring Program Comprehension: A Large-Scale Field Study with Professionals," *IEEE Trans. Software Eng.*, vol. 44, no. 10, pp. 951-976, 2018.
- [2] A. J. Ko, B. A. Myers, M. J. Coblenz, and H. H. Aung, "An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks," *IEEE Trans. Software Eng.*, vol. 32, no. 12, pp. 971-987, 2006.
- [3] R. Minelli, A. Mocci, and M. Lanza, "I Know What You Did Last Summer: An Investigation of How Developers Spend Their Time," *IEEE ICPC*, 2015, pp. 25-35.
- [4] R. Brooks, "Towards a Theory of the Comprehension of Computer Programs," *Int. J. Man-Machine*

Studies, vol. 18, no. 6, pp. 543-554, 1983.

[5] V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos, "PyCG: Practical Call Graph Generation in Python," *Proc. ICSE*, 2021, pp. 1646-1657.